Large Language Model (LLM) Training - Intro - final

Scope:

- Only GPT-style LLMs (decoder-only). We ignore non-GPT families like encoder-decoder (seq2seq), encoder only (e.g. rerankers) or diffusion models
- Ignore multimodal LLM's. It's pretty much the same as standard GPT-style LLM with additional encoder for modality (audio, image etc).

Approach:

- Just enough detail to get intuition but not in-depth exploration. It's impossible to cover so many topics in depth but reader should have good mental model how the "bits" fit together and can dive deeper into each of the areas.
- Avoid complex mathematical notation (not needed at this level) and use analogies or intuitions instead. Every term is introduced first before used (no blasting with autoregression, topP/K, KV cache, transformer without explaining what it is)
- As this is training oriented we will focus on what to pay attention too - what is important for training focusing on engineering aspects.
- Analogies/mental models are geared towards software engineers as they are primary audience.

0. DNNs — the minimum you need (skip if you already know it)

Before we touch Transformers, we need a common gut feel for **neural nets**, **forward pass**, **loss**, and **backpropagation**—and why **matrix multiplication** keeps showing up everywhere. We'll do this with a toy feed-forward network, the simplest model there is. Once this clicks, swapping the simple blocks for Transformer blocks later will feel easier.

STEP 1: A SINGLE NEURON

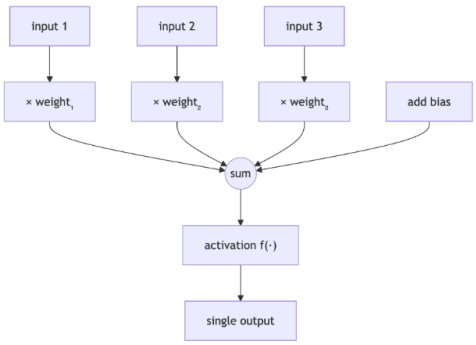
What a neuron has:

- **Weights** — one weight per input. Each weight says how much that input matters.
- **Bias** — a small constant added after the weighted sum; lets the neuron "turn on" even when inputs are small.
- **Activation function** — a simple "squish" applied to the sum (e.g., ReLU = "clip negatives to 0", GELU = smooth squish). This is the only non-linear part.

A neuron is just an operation:

- take a few inputs (numbers)
- multiply each by its own weight
- add a small constant called **bias**
- pass the sum through a simple **activation** (a "squish" like ReLU)

That's it. No memory, no magic—just "multiply → add → squish → output."

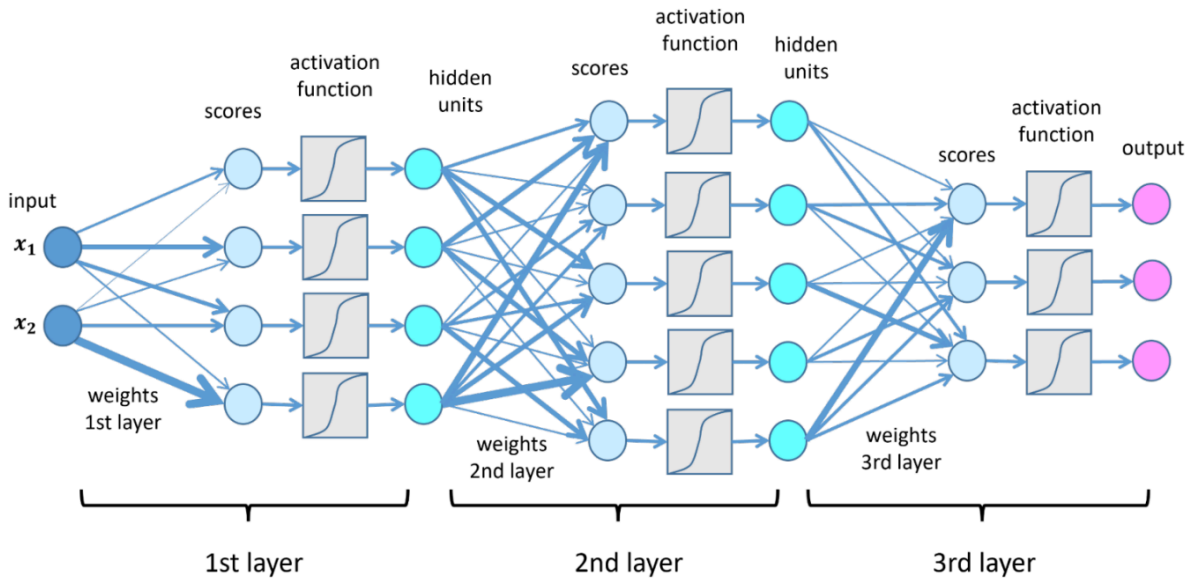


Below is the code version of what is happening in the diagram.

```python
def relu(x):      # ReLU - just remove everything below 0. if e.g. -1 return 0.
    return x if x > 0 else 0.0

def neuron_output(inputs, weights, bias, activation=relu):
    # weighted sum (dot product) + bias
    s = sum(i * w for i, w in zip(inputs, weights)) + bias
    # squish it with an activation
    return activation(s)

inputs  = [0.8, 0.2, 1.0]
weights = [1.5, -0.5, 0.3]
bias    = -0.2

y_relu     = neuron_output(inputs, weights, bias, activation=relu)     # -> 1.2

print(y_relu, y_sigmoid)
```

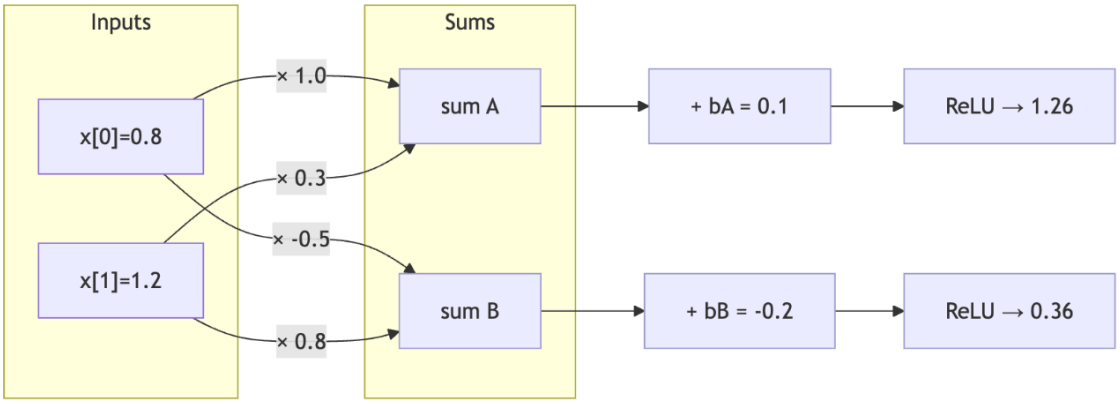
STEP 2 — CONNECT NEURONS → LAYERS → NETWORK

Put several neurons side-by-side. Each sees the same inputs but has its own weights/bias. That's a **layer**. Stack layers so the outputs of one become the inputs to the next. That stack is your **neural network**.



Circles = values moving forward (often called activations). Gray boxes = activations applied per neuron. Arrows = learned weights. Left = inputs; right = outputs.

A **layer** is just many neurons side-by-side, all doing "multiply → add → squish." If you pack all the neuron weights into one big table `W` and your inputs into a vector `x`, the whole layer becomes one operation:




```
x = [0.8, 1.2]          # 2 inputs (input dimensions - 2)

W = [[ 1.0, -0.5],    # weights
     [ 0.3,  0.8]]
#  shape: (in_dim=2) × (out_dim=2) -
#  Column 0 (neuron A): [1.0, 0.3]
#  Column 1 (neuron B): [-0.5, 0.8]
b = [0.1, -0.2]         # one bias per neuron

raw = x @ W                   # = [0.8*1.0 + 1.2*0.3,  0.8*(-0.5) + 1.2*0.8]
     = [1.16, 0.56]

raw_plus_bias = [1.16+0.1, 0.56-0.2]  = [1.26, 0.36]
y = ReLU(raw_plus_bias)               = [1.26, 0.36]
```

So the whole layer is just: **multiply (x @ W)** → **add bias (+ b)** → **squish (ReLU)** → outputs. If you stack layers, the next layer would take `[1.26, 0.36]` as its input vector and repeat the exact same pattern.

This is where GPU's come in. GPUs crush this because matrix multiply is embarrassingly parallel: thousands of tiny multiply-adds that can be done at once. Modern GPUs/TPUs also have:

- **Fused multiply–add** units (do `a*b + c` in one step). This is the multiple weight and add to "running sum".
- **Tensor Cores**. Each one computes a *small* matrix multiply (a "tile," e.g., 16×16) very fast, usually in lower precision (fp16/bf16/TF32/int8) while **accumulating** in higher precision (often fp32) for accuracy.
- **Fast on-chip memory** to reuse those tiles without round-tripping to slow DRAM.

A layer is just many neurons at once, which the GPU treats as one matrix multiply: `X @ W`, then add bias and apply the activation. The hardware breaks that big multiply into small square "tiles," pulls each tile pair into fast on-chip memory, and feeds them to **Tensor Cores**—tiny engines that blast through lots of fused multiply-adds while accumulating the result. When writing the output back to memory, kernels usually **fuse** the bias add and activation so you don't make extra passes. That's your neuron math, just done thousands of times in parallel.

Throughput then comes from **shape and reuse**. A decent **batch** turns `X` into a tall matrix so each loaded weight tile from `W` can be reused across many rows before it leaves the chip. Dimensions that are friendly to tiles (often multiples of 8/16) keep Tensor Cores fully busy; padding is often faster than "perfectly tight" shapes. Run in mixed precision (bf16/fp16) so Tensor Cores hit peak speed while accumulating in fp32 for accuracy. In short: feed the GPU **big, well-shaped matrices** and you get massive throughput—that's why training and inference code are obsessed with **batching and tiling**.

Why the activation function matters

If you stacked layers **without** an activation (like ReLU) —just multiplication and sum over and over—the whole stack collapses to a **single linear transform**. No matter how many layers you add, the model can only draw straight lines in high-dimensional space. It can't learn even simple curved rules (like XOR), let alone language.

The activation is the tiny nonlinearity that breaks that limitation. Common ones:

- **ReLU**: `max(0, x)`. Simple, fast, stable; can "die" (output 0) if pushed negative too often.
- **GELU**: a smooth curve; widely used in GPT-style models because it plays nicely with deep stacks.
- **SiLU / Swish**: another smooth option; some GPTs use **SwiGLU** (a gated variant) in the MLP.

Without this "squish," deeper network just means slower, not smarter.

STEP 3 - TRAINING DNNS

Training is just a loop that tweaks numbers until the model's guesses match targets. You start with **inputs** (e.g., `[0.8, 0.2]`) and a **target** (say `1.0`). The **forward pass** runs the network's math (multiply → add → squish) to make a prediction—maybe `0.6`. A **loss** turns "how wrong" into one number (our toy example has a single output, but the network could have more) —for our toy case, $(0.6 - 1.0)^2 = 0.16$. The **gradient** is simply the local "which way is downhill?" for each weight —if increasing a weight would raise loss, its gradient is positive; if it would lower loss, it's negative. Autodiff/backprop is the framework replaying the forward pass backwards to compute those downhill directions automatically—you don't do calculus. The optimizer **step** (SGD/AdamW) then nudges every weight a tiny bit against its gradient; the **learning rate** is how big that nudge is. Clear the gradients and repeat on the next batch.

That's it: **forward** → **loss** → **backprop** → **step** → **repeat**. Next time through, the same input might predict `0.65`, then `0.72`… loss falls, and the model improves. When we switch to LLMs, this loop is identical—only the inputs become token sequences and the targets become the correct next token

Why training needs more memory (and more compute) than inference

Forward pass is cheap-ish: one layer does "multiply → add → squish." In matrix form that's one big `X @ W` (plus bias and activation). Inference mostly just does that once per layer and moves on.

Backward pass adds two big jobs: after you compute the loss, training runs the graph **backwards** to figure out how to tweak every weight. For a dense layer, backward has to (a) compute a **nudge for each weight** (think: one tiny number per weight that would reduce loss) and (b) compute an **error signal for the previous layer** so it can learn too. Intuitively, that's about **two more matrix multiplies** of similar size to the forward one, plus some small element-wise work (like zeroing gradients where ReLU was off). That's why a train step costs ~**2–3×** a forward pass in FLOPs.

Why memory explodes: backward needs "breadcrumbs" from forward.

- You must **keep the layer's inputs** so you can form weight nudges ("input × how wrong that neuron was").
- You must **keep pre-activations or masks** (e.g., which ReLUs were on) to know where gradients should flow.
- In Transformers you also keep per-token representations, and attention needs some intermediates (modern kernels recompute some pieces to save memory, but the pressure is still real).

On top of activations, training stores:

- **Weights** (same as inference),
- **Gradients** (one tensor the same size as the weights),

- **Optimizer state** (AdamW keeps two extra tensors per weight for momentum/variance).

Rule-of-thumb footprint on a single GPU becomes: **weights + grads + optimizer state ≈ 3–4× model size**, **plus activations**, and activations scale with **batch × sequence length × hidden width × layers**. That last product is why long contexts in LLMs are memory killers.

Common knobs are just compute↔memory trades: **mixed precision** (store activations in bf16/fp16), **activation checkpointing** (don't store, recompute during backward), **gradient accumulation** (simulate a big batch over several micro-steps), and **optimizer sharding** (don't keep all optimizer tensors on every device).

How distributed training works

Data parallel (many identical workers).
Copy the whole model to each GPU. Split the input data; each GPU runs forward/backward on its slice, then everyone averages gradients (an all-reduce) and applies the same weight update. Super simple and scales well, but each GPU must fit the whole model + optimizer + its activations (not really possible for larger LLMs).

Tensor/model parallel (one giant matrix multiplication, sliced).
When a single layer is too big, split the layer's weight matrix across GPUs (e.g., split columns/rows). Each GPU holds a shard, computes its piece of `X @ W`, and you all-gather / all-reduce between layers to stitch results. Mental model: one huge matrix multiply broken into blocks across devices. Great for very wide layers, but adds comms between layers.

Pipeline parallel (assembly line of layer chunks).
Split the stack of layers into stages across GPUs. Push micro-batches through like an assembly line so multiple stages work at once. This cuts per-GPU activations (each stage only stores its slice).

More resources to learn:

- https://www.3blue1brown.com/?v=neural-networks - 3Blue1Brown But what is a Neural Network?

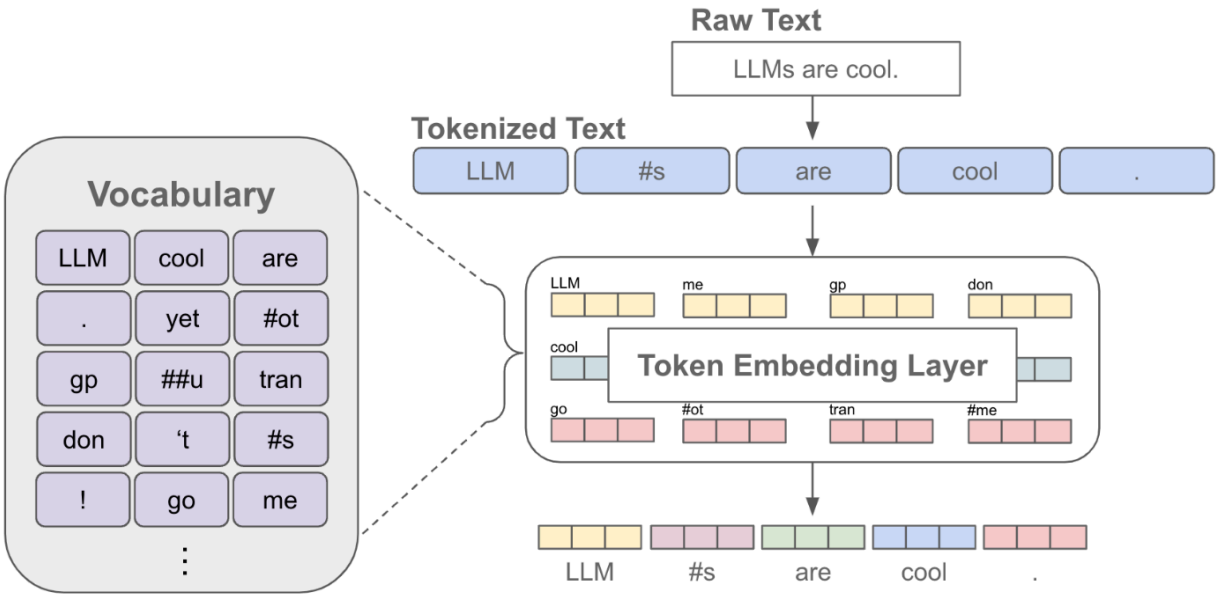
1. LLMs at 10,000 ft

Now that you understand the neural network basics from Section 0, let's see how GPT-style models work. A **language model** is simply a neural network trained to predict the next token (think word or word-piece) in a sequence. You feed it "The weather today is" and it might predict "sunny" with 60% confidence, "cloudy" with 25%, "terrible" with 10%, etc. The core insight is that this simple task—predicting what comes next—captures an enormous amount of knowledge about language, reasoning, and the world. Just like how the feed-forward networks from previous section used matrix multiplies (`X @ W`) to transform inputs, language models do the same thing, but the inputs are sequences of tokens and the network has learned to compress human knowledge into its weight matrices.

Transformers take those same neural network building blocks you just learned—the "multiply → add → squish" layers—and add **attention** mechanisms between them. Attention is the ability to look back across the entire input sequence and decide what matters for predicting the next token. Without attention, a model can only see a small local window (like the last few words). But with attention, the model can connect "The bank" at the beginning of a sentence to "deposit" 20 words later, or distinguish between "bank" (financial institution) and "bank" (river's edge) based on context from anywhere in the input. Here's why context wins: the phrase "...the bank" could be followed by "deposit" if the earlier context mentioned money (`"I need to go to the bank to make a deposit"`), but "erosion" if it mentioned a river (`"The flooding damaged the bank, causing severe erosion"`). Same local phrase, completely different next tokens—the transformer's attention mechanism lets it scan the entire prefix, weight what's relevant, and make the right prediction.

2. From words to tokens (vocabulary and tokenization)

Language models predict the "next token"—but what exactly is a token? A token is the atomic unit the model works with, not quite a character and not quite a word. Think of tokens as entries in a vocabulary table, where each entry gets a unique integer ID. Just like Section 0 showed how neural networks need numbers to do matrix math, language models need text converted to integers before they can multiply, add, and squish their way to predictions



Why not characters or whole words? Character-level means learning spelling from scratch and painfully long sequences. Word-level means 170,000+ vocabulary entries just for common English, plus technical terms, before even considering other languages. Tokens split the difference: common words like "the" get single tokens, rare words break into pieces like "anti" + "dis" + "establishment". This keeps vocabulary manageable (30K-100K tokens) while handling any text.

How tokenization works: Modern tokenizers like BPE (Byte Pair Encoding) learn from massive text corpora. They start with characters and repeatedly merge frequent pairs—if "th" appears millions of times, it becomes one token. After thousands of merges, you get a vocabulary where common patterns are single tokens. At runtime, the tokenizer greedily matches the longest tokens it can find:

```
"Full funnel advertising optimization" → ["Full", " funnel", " advertising", " optimization"] # 4 tokens
"Full funnel advertising for ASIN B08XYZ123Q" → ["Full", " funnel", " advertising", " for", " AS", "IN", " B", "08", "XY", "Z", "123", "Q"] # 12 tokens!
```

That 10-character ASIN exploded into 7 tokens because the tokenizer never saw "B08XYZ123Q" during training. This is why models handle 2,000 words of prose but only 500 product IDs in the same context window.

Context windows: "128K tokens" means the model can process 128,000 integers at once. But your mileage varies:

- Normal prose: ~750 words per 1000 tokens
- Code: ~500 words per 1000 tokens
- IDs/URLs/base64: ~200 characters per 1000 tokens

Important for us: operating on random numbers (like identifiers and raw numbers) eats a LOT of tokens.

The vocabulary training pipeline:

1. **Collect corpus** (1TB of text like blog posts, help articles etc)
2. **Build vocabulary** (BPE learns ~50K tokens from frequency patterns - you tell BPE how big vocabulary should be)
3. **Tokenize everything** (convert all text to integers—once, before training)
4. **Save integers** (these sequences become your actual training data)
5. **Train** (model loads pre-tokenized integers, never raw text)

Important for engineering:

Lock your tokenizer. Once training starts, never change it. Token 42 meaning "cat" during training but "quantum" during inference = dead model. Version everything.
Pre-tokenize for pre-training. Convert text to integers once and save them. Loading integers is fast; tokenizing every epoch wastes GPU cycles as tokenization is CPU/memory heavy. For large-scale training, split tokenized data into 10GB shards that workers read in parallel. (Note: For smaller fine-tuning datasets, you might tokenize on-the-fly while experimenting.)
Design for your domain. Code model? Make keywords single tokens. Medical? Add drug names. Advertising? Make sure we have tokens for common terms. But remember: 100K vocabulary × 4096-dim embeddings = 400M parameters just for the lookup table (Section 3).

3. Embeddings and positional information

Section 2 gave us integers—token IDs like `[13295, 29276, 13172, 23669]` for "Full funnel advertising optimization". You could feed these integers directly to a neural network, but they'd be meaningless numbers. Token 13295 ("Full") and 13296 (maybe "full") would seem as different as 13295 and 49999. We need representations that capture **semantic meaning**—where similar concepts are close together and different concepts are far apart.

The embedding table learns meaningful vectors for each token. It's a giant matrix with 50,000 rows (one per token in vocabulary) and 4,096 columns (the model's hidden dimension). Token ID 13295 means "grab row 13295":

```
# Embedding table: [vocab_size × hidden_size]
embedding_table = torch.randn(50000, 4096)  # Initialized random, learned during trair

token_ids = [13295, 29276, 13172, 23669]  # "Full funnel advertising optimization"
vectors = embedding_table[token_ids]  # Shape: [4, 4096], 4 embeddings, one for each t
```

The magic happens during training. The model learns to place semantically similar tokens nearby in vector space. After training:

- **"cat" and "kitten"** have high cosine similarity (vectors point similar directions)
- **"cat" and "fridge"** have low similarity (vectors point different directions)
- **Distance = meaning**: The vector distance measures semantic difference

This isn't programmed—it emerges from predicting next tokens. The model learns that "cat" and "kitten" appear in similar contexts, so their embeddings converge to nearby points in the 4096-dimensional space.

One embedding per token, not per word. When words split into multiple tokens, each piece gets its own embedding:

```
"unbreakable" → ["un", "break", "able"]  # 3 tokens, 3 embeddings
embeddings = [
    embedding_table[8241],   # "un" - negation prefix
    embedding_table[9876],   # "break" - core concept
    embedding_table[1234]    # "able" - capability suffix
```

```
]
```

There's no single "unbreakable" vector. The model sees three vectors in sequence and learns through attention (Section 4) how they combine to form meaning.

Even "nonsense" tokens learn useful patterns. Take the fragment "prot" that appears in many words:

- "protein" → ["prot", "ein"]
- "prototype" → ["prot", "otype"]
- "protect" → ["prot", "ect"]

The embedding for "prot" becomes a statistical average of all its contexts—somewhat bio/science-leaning but general enough for prototype/protect. It learns to be a "technical prefix" embedding. Similarly, tokens like "##XY" that appear in product IDs learn to represent "middle of an identifier" even though they have no standalone meaning.

But embeddings alone lose position information. Consider:

- "The cat chased the mouse"
- "The mouse chased the cat"

Same tokens, different meaning! Without position information, attention can't distinguish them—the embedding for "cat" is identical whether it's the chaser or the chased. You might think "just add the position as another dimension"—but raw position numbers create problems. Position 1 gets value 1, position 100 gets value 100—now late positions have huge values that dominate the embedding. Plus, the model needs to learn that "2 tokens apart" means the same relationship whether that's positions 1→3 or positions 98→100. Raw numbers don't encode these relative patterns naturally.

Rotary Position Embeddings (RoPE) encodes position through rotation. Instead of adding position vectors, RoPE rotates embedding vectors based on their position. This means no additional memory needed for positional data.

Think of it like multiple clocks running at different speeds—slow clocks for capturing document-level patterns, fast clocks for adjacent word relationships. The attention mechanism can look at these rotation patterns to figure out relative positions.

Sparse updates save computation. Each batch only uses ~1,000 unique tokens out of 50,000. Smart implementations only compute gradients for accessed embeddings, but the full table stays in memory.

RoPE and long contexts. While RoPE adds no parameters, the rotation frequencies are tuned for your training context length. Train with 2K tokens but deploy at 128K? The high-frequency rotations can destabilize. Recent work (RoPE scaling, YaRN) adjusts frequencies for longer contexts.

Engineering: Embedding tables are prime targets for quantization during inference. They're memory-bandwidth bound (just lookups), so using int8 or int16 instead of fp32 doubles effective bandwidth with minimal quality loss.

4. The Transformer Block (the unit we stack N times)

Section 3 gave us embeddings—a vector for each token with position information baked in. But there's a problem: each token's vector is identical regardless of context. The vector for "bank" is the same whether the sentence is about rivers or money. Tokens need to look at each other to figure out what they actually mean in context. That's what a transformer block does.

WHAT GOES IN AND WHAT COMES OUT

A transformer block takes all your token vectors at once—your entire context window—and processes them together:

python

```
# Input: ALL tokens in your sequence
input_tokens = [...] # Shape: [2048 tokens × 4096 hidden_size]
# Output: Same tokens, but now context-aware
output_tokens = transformer_block(input_tokens)  # Still [2048 × 4096]
# Key point: dimensions don't change!
# This lets us stack blocks: output of block 1 → input to block 2
```

Every token goes in, every token comes out, but now each token "knows about" the others.

INSIDE A TRANSFORMER BLOCK

A transformer block is surprisingly simple—just two operations with some wiring:

```
def transformer_block(tokens):  # tokens: [seq_len × hidden_size]
# Operation 1: Attention - let tokens look at each other
    tokens_normed = layer_norm(tokens)               # Normalize for stability
    attention_out = attention(tokens_normed)         # THE EXPENSIVE PART
    tokens = tokens + attention_out                  # Residual connection
# Operation 2: MLP - process the enriched tokens
    tokens_normed = layer_norm(tokens)
    mlp_out = mlp(tokens_normed)                    # Just a 2-layer network!
    tokens = tokens + mlp_out                        # Another residual
return tokens  # Same shape as input: [seq_len × hidden_size]
```

That's it. Attention → MLP → next block. The residual connections (the `tokens +` part) mean each block adds to the running representation rather than replacing it—like Git commits adding changes rather than rewriting everything.

THE MLP IS JUST THE NETWORK FROM SECTION 0

Remember the 2-layer network from Section 0? That's literally what the MLP (Multi-Layer Perceptron) is—despite the fancy name, it's just 2 layers:

```
def mlp(x):  # x is [seq_len × hidden_size]
    # Layer 1: expand to 4× size
    x = x @ W_up + bias         # [2048 × 4096] @ [4096 × 16384] = [2048 × 16384]
    x = gelu(x)                 # Activation - same "squish" concept from Section 0
    # Layer 2: contract back
    x = x @ W_down + bias       # [2048 × 16384] @ [16384 × 4096] = [2048 × 4096]
    return x
```

ATTENTION: WHERE COMPUTE AND MEMORY EXPLODE

Attention lets each token look at all previous tokens to gather relevant information. The mechanism is elegant, but creates a massive scaling problem.
I don’t understand exactly “why” it works but below is most of what you need to understand about scaling and problems.
This is the best video on the topic to dive deeper: Attention in transformers, step-by-step | Deep Learning Chapter 6

```
def attention(tokens):  # tokens: [seq_len × hidden_size]
```

```
    # Three learned weight matrices - these are the parameters
    W_query = [...]  # [4096 × 4096] = 16.8M parameters
    W_key = [...]    # [4096 × 4096] = 16.8M parameters
    W_value = [...]  # [4096 × 4096] = 16.8M parameters

    # Transform the input three different ways
    Q = tokens @ W_query  # "What each token is looking for"
    K = tokens @ W_key    # "What each token offers"
    V = tokens @ W_value  # "The actual information to share"

    # All three still [seq_len × hidden_size]
    # THE PROBLEM: compute how much each token cares about each other token
    scores = Q @ K.transpose()  # [seq_len × seq_len] ← QUADRATIC!
    # Example at 16K context:
    # Compute: 16K × 16K × 4K = 1 trillion multiplications
    # Memory: [16384 × 16384] = 268M numbers = 536MB in fp16
    # Both scale with seq_len²!

    weights = softmax(scores)     # Convert to probabilities
    output = weights @ V           # Mix information based on attention

    return output  # Back to [seq_len × hidden_size]
```

Let's be precise about what "quadratic" means here:

- Creating Q, K, V: O(seq_len × hidden_size²) - scales linearly with context
- Computing scores: O(seq_len² × hidden_size) - scales quadratically
- Storing scores: O(seq_len²) memory - also quadratic

THE QUADRATIC SCALING - THIS IS THE MAIN PROBLEM

Both compute and memory scale quadratically, but hit limits at different points:

	Context	Score Matrix	Compute	Memory	Time @ 300 TFLOPS
1	1K tokens	1K × 1K	4B ops	2 MB	0.01ms
2	8K tokens	8K × 8K	262B ops	128 MB	0.9ms
3	32K tokens	32K × 32K	4.4T ops	2 GB	14ms
4	128K tokens	128K × 128K	~70T ops	32 GB	220ms

Memory bandwidth becomes the bottleneck before compute does. Modern GPUs can do 300+ TFLOPS but only move 2TB/s of memory. When you're constantly moving those huge attention matrices in and out of memory, the GPU spends most time waiting for data, not computing.

HOW FLASH ATTENTION SAVES THE DAY

Flash Attention (the engineering breakthrough that makes modern LLMs possible) never creates that giant attention matrix:

```
# Standard attention: materialize everything
scores = Q @ K.T              # Creates [seq_len × seq_len] in memory
attention = softmax(scores)  # Read huge matrix, write it back
output = attention @ V       # Read huge matrix again
```

```
# Flash Attention: compute in chunks that fit in SRAM (fast on-chip memory)
# SRAM: 20MB but 10× faster than main GPU memory (HBM)
for chunk_q in chunks(Q):
    for chunk_k, chunk_v in chunks(K, V):
        # These chunks fit in 20MB SRAM!
        chunk_scores = chunk_q @ chunk_k.T
        # Process entirely in fast memory
# Accumulate output without storing full matrix
```

Flash Attention actually does MORE compute (recomputes in backward pass) but runs 2-4× faster because it minimizes slow memory transfers. It's a perfect example of how "optimal" algorithms (less compute) can be slower than hardware-aware algorithms.

MULTI-HEAD: DOING ATTENTION IN PARALLEL SLICES

Instead of one big attention computation, we run multiple smaller ones in parallel:

```
hidden_size = 4096
num_heads = 32
head_size = 4096 / 32 = 128  # Each head gets 128 dimensions
# The weight matrices are the same total size, just logically divided:
# Instead of one [4096 × 4096] attention
# We have 32 parallel [128 × 128] attentions
# Different heads learn different patterns:
# - Head 1 might focus on grammar
# - Head 2 might track entities
# - Head 3 might follow dependencies
# ... all learning automatically from data
```

This doesn't reduce compute or memory—the total operations are identical. It just organizes the computation better and lets different heads specialize.

[TANGENT - INFERENCE TIME] KV CACHING: TRADING MEMORY FOR COMPUTE

The K and V matrices we just computed in attention have a useful property: they only depend on their input tokens, not on what comes after. This means we can cache and reuse them—critical for making generation fast enough to be usable.
Generation cache: When generating "The cat sat on the mat" one token at a time, we need all previous K,V vectors for attention. Without caching, we'd recompute K,V for "The", then "The cat", then "The cat sat"—quadratic work. With caching, compute each token's K,V once and reuse.
Prompt cache: System prompts repeated across requests (like "You are a helpful assistant...") can have their K,V cached and shared. First request computes and stores, later requests just load from cache.
The memory cost per token:

```
# Must cache at EVERY layer:
32 layers × 2 vectors (K,V) × 4096 dims × 2 bytes = 524KB per token

32K context = 17GB KV cache
```

```
128K context = 67GB KV cache (approaching model size!)
```

Production systems must compress this:

- Grouped-Query Attention (GQA): Multiple heads share K,V (8× reduction)
- Quantization: INT8 or INT4 instead of FP16 (2-4× reduction)
- Sliding window: Only cache recent tokens (fixed size)

That "128K context" isn't just about compute—it's about fitting 67GB of KV cache somewhere. Most context limits are actually memory limits. So if the model "performs bad" at inference it could be due to compression of KV cache.

TRAINING: SAME GRADIENT DESCENT AS SECTION 0

All these weight matrices (W_query, W_key, W_value, W_up, W_down) learn through the exact same backpropagation from Section 0:

```
# Forward pass
output = transformer_block(input)
loss = cross_entropy(output, next_token_target)

# Backward pass - compute gradients for ALL weights
gradients = backprop(loss)
# Update all weights identically (SGD or AdamW)
W_query -= learning_rate * gradients['W_query']
W_key   -= learning_rate * gradients['W_key']
W_value -= learning_rate * gradients['W_value']
W_up    -= learning_rate * gradients['W_up']
W_down  -= learning_rate * gradients['W_down']
```

No special training procedure. The attention mechanism is just architecture—like choosing ReLU vs GELU. The learning is still plain gradient descent on weight matrices.

TRAINING TIME CONCERNS : CAUSAL MASKING

During training, we process entire sequences at once for efficiency. But each position must only see previous positions, not future ones—otherwise the model cheats by looking at what it's supposed to predict.

```
# Training on: "The cat sat on"
# Position 3 ("sat") must predict position 4 ("on")
# Without masking: position 3 can see "on" directly - cheating!
# The mask blocks future positions:
Position 1 can see: [The]
Position 2 can see: [The, cat]
Position 3 can see: [The, cat, sat]    # Can't see "on"!
Position 4 can see: [The, cat, sat, on]
# Implementation: Set future attention scores to -infinity
scores = Q @ K.T
mask = torch.triu(torch.ones_like(scores) * -inf, diagonal=1)
scores = scores + mask  # Future positions → -inf → 0 after softmax
```

Without causal masking, training breaks completely—the model learns to copy instead of predict. At inference this doesn't matter (generating one token at a time anyway), but it's mandatory for parallel training.

5. Stacking blocks into a model

Real models stack transformer blocks dozens of times—Qwen3-32B has 64 layers, LLaMA-70B has 80. Each block has the same structure from Section 4, just different learned weights.

Depth scales linearly, not quadratically like attention - FIX MATH:

```
# Memory and parameters per block (using Qwen3-32B as example):
# hidden_size = 4096, intermediate_size = 22016 (5.4× instead of usual 4×)
# num_layers = 64 (not 32)

params_per_block =
    3 * (4096 * 4096)              # Q, K, V matrices = 50.3M
    + (4096 * 22016)                # MLP up = 90.2M
    + (22016 * 4096)                # MLP down = 90.2M
    # Total per block: ~230M parameters

# 64 blocks × 230M = 14.7B params
# Plus embeddings [151936 × 4096] = 622M params
# Model total varies based on architecture details (GQA, tied embeddings, etc.)

# Training memory per block:
# - Activations: batch_size × seq_len × 4096 × ~10
# - Gradients: same size as parameters
# - Optimizer state: 2× parameters (for AdamW)
```

Residual connections (the `tokens = tokens + ...` from Section 4) are mandatory for deep networks. Without them, gradients vanish—multiply 0.9 by itself 94 times and you get 0.00000012. The residual creates a direct path for gradients to flow backward, bypassing the transformations. Modern models do 90+ layers; the original transformer could only do 6. Depth typically stops around 80-100 layers because:

- Activation memory: Must store intermediate values for all layers during backward pass
- Diminishing returns: Most improvement comes from first 50-60 layers
- Optimization difficulty: Deeper models need more careful hyperparameter tuning

With gradient checkpointing, you can trade compute for memory—recompute activations during backward pass instead of storing them. This roughly doubles compute time but can cut memory by 10×, making very deep models possible on limited hardware.

6. Lets now get the next word - logits

After 32 transformer blocks, we have a refined hidden state for each token position. But we need to predict an actual word. The **hidden state** (fancy word for output from transformer) for the last token is a 4096-dimensional vector full of abstract features. How does this become "the next word is 'cat' with 73% confidence"?

The "language model head" is just one more matrix multiplication—no attention, no fancy architecture:

```
# Final hidden state from last transformer block
final_hidden = transformer_output[-1]  # Shape: [4096]
```

```
# LM head is just a matrix: [hidden_size × vocab_size]
lm_head_weights = [...]  # [4096 × 151936] = 622M parameters!
# Project to vocabulary
logits = final_hidden @ lm_head_weights  # Shape: [151936]
# logits[42] = "score" for token 42 being next
```

Those 151,936 numbers are called "**logits**"—raw scores for each token in your vocabulary. Higher logit = model thinks this token is more likely. But logits aren't probabilities—they can be negative, huge, whatever. Token 42 might have logit 8.3, token 100 might have -2.1.

Why can't we use logits directly? We need to sample from them—pick one token based on how likely each one is. But you can't sample from arbitrary numbers. If token A has logit 8.3 and token B has -2.1, what's the probability of picking A? The raw numbers don't tell us. We need actual probabilities that sum to 1.0 so we can use them as weights for random selection.

Softmax converts arbitrary scores into probabilities. It exponentiates each logit (e^x) then divides by the sum of all exponentiated values. In practice, implementations subtract the max logit first to prevent numerical overflow—otherwise e^100 would explode to 10^43. The exponential amplifies differences: a logit difference of 2 becomes a probability ratio of about 7:1. This helps the model be confident about likely tokens while keeping some probability for alternatives.

Now we have probabilities and can actually sample. But always picking the highest probability token (greedy decoding) produces boring, repetitive text. We need controlled randomness.

Temperature controls randomness by scaling logits before softmax. When you divide logits by temperature:

- **Low temperature (0.1)**: Divide by 0.1 = multiply by 10. Logits [2.0, 1.0] become [20.0, 10.0]. After softmax, e^20 vs e^10 gives the first token 99.995% probability. Nearly deterministic.
- **High temperature (2.0)**: Divide by 2.0 = halve the logits. Logits [2.0, 1.0] become [1.0, 0.5]. After softmax, e^1 vs e^0.5 gives a 62/38 split. More random.

Temperature controls how much softmax amplifies differences. Low temperature = more amplification = winner takes almost all. High temperature = less amplification = more even distribution.

But pure temperature sampling can still pick nonsense tokens that have 0.001% probability. We need to restrict our choices. There are two common methods (you typically use one or the other, though they can be combined):

- **Top-k sampling** only considers the k highest probability tokens. Set k=10, and you only sample from the 10 most likely next words, ignoring the other 151,926 tokens entirely. After filtering to top-k, you renormalize those k probabilities to sum to 1.0 and sample. Simple and predictable.
- **Top-p (nucleus) sampling** is smarter—it adapts to the model's confidence. Instead of always taking the top 10 tokens, it takes however many tokens are needed to cover 90% (or whatever p you set) of the probability mass. When the model is confident (one token has 85% probability), top-p=0.9 might only keep 2-3 tokens. When uncertain (many tokens with 2-5% each), the same p=0.9 might keep 50+ tokens.

The key point: both methods filter first, then randomly sample **one token from what remains**. The randomness comes from weighted random selection—higher probability tokens are more likely to be picked, but not guaranteed.

This is why LLMs non-deterministic. When you randomly pick from a probability distribution, you get different results each time. Set temperature to 0 (greedy decoding—always pick the max), and the model becomes mostly deterministic. "Mostly" because floating-point operations on GPUs can still have tiny variations, but for practical purposes, greedy decoding gives you the same output every time

Logit masking gives you precise control over what tokens can be generated. Before applying softmax, you set unwanted

tokens' logits to -infinity, giving them zero probability:

```
# Example: Force JSON output to only use valid tokens
if expecting_json_key:
    # Only allow alphabetic characters and quotes
for i, token in enumerate(vocab):
        if token not in valid_json_key_tokens:
            logits[i] = float('-inf')
# Example: Prevent repetition
recent_tokens = [1234, 5678, 9012]  # Last 3 generated tokens
for token_id in recent_tokens:
    logits[token_id] = float('-inf')  # Can't repeat recent tokens
# After masking, continue with normal softmax + sampling
probs = softmax(logits)  # Masked tokens get 0 probability
```

This is how structured generation works—forcing the model to produce valid JSON, SQL, or other formatted output by masking invalid tokens at each step. It's also used for repetition penalties, content filtering, and constraining outputs to specific vocabularies.

7. Training Loop (End-to-End)

7.1 WHAT IS A BATCH AND WHY WE NEED IT

A batch is simply multiple training examples processed together. Think of it like grading papers—you could grade one at a time, or grab a stack and process them together. The second way is more efficient for both humans and GPUs.
Let's see this concretely. Say we're training on three sentences:

- "The cat sat on the mat" (6 tokens)
- "Machine learning is powerful" (4 tokens)
- "Amazon builds AI" (3 tokens)

These become token sequences of different lengths. But GPUs need fixed-size tensors—remember from Section 0, matrix multiplication needs rectangular matrices, not ragged arrays. So we pad shorter sequences with a special `[PAD]` token to make them all the same length:

```
batch = [
    [546, 3857, 4270, 389, 264, 5259, 0, 0],  # "The cat sat on the mat" + 2 pads
    [18668, 6975, 374, 8147, 0, 0, 0, 0],     # "Machine learning is powerful" + 4 pad
    [26948, 22890, 15592, 0, 0, 0, 0, 0],     # "Amazon builds AI" + 5 pads
]# Shape: [3 sequences, 8 tokens each]
```

Each row is one complete training example. The GPU processes all three rows simultaneously through the transformer blocks from Section 4.
Why batching matters: Remember from Section 0 that GPUs have thousands of cores that can do parallel multiply-adds? With one example, most cores sit idle. With a batch of 64 examples, those cores all work simultaneously. Batch size 1 might achieve 5% GPU utilization. Batch size 64 can hit 90%.

7.2 THE PARALLEL PREDICTION INSIGHT - HOW TRANSFORMERS LEARN FROM EVERY TOKEN

Here's the key insight that makes transformers revolutionary: when we feed in "The cat sat on the mat", we don't just learn to predict "mat" from the entire sequence. Every position learns to predict its next token, all in one forward pass.
Think about what happens after the forward pass through all transformer blocks:

- Position 0 has encoded "The" → predicts "cat"
- Position 1 has encoded "The cat" → predicts "sat"
- Position 2 has encoded "The cat sat" → predicts "on"
- Position 3 has encoded "The cat sat on" → predicts "the"
- Position 4 has encoded "The cat sat on the" → predicts "mat"

All five predictions happen simultaneously! The causal mask from Section 4 ensures each position only sees earlier tokens (no cheating), but the computations are parallel, not sequential.

```
# After forward pass through transformer blocks:
hidden_states = model(tokens)  # Shape: [seq_len=6, hidden_size=4096]
# Each position has a different representation:
# hidden_states[0] = encoding of just "The"
# hidden_states[1] = encoding of "The cat"
# hidden_states[2] = encoding of "The cat sat"
# ... etc
# Project to vocabulary to get predictions:
logits = hidden_states @ lm_head  # Shape: [6, vocab_size=151936]
# Now logits[0] contains prediction for what comes after "The"
# logits[1] contains prediction for what comes after "The cat"
# ... all computed in parallel!
```

This is why transformers replaced RNNs. From a 100-token sequence, RNNs got 1 training signal after 100 sequential steps. Transformers get 99 training signals in one parallel step.

7.3 HOW THE COMPLETE TRAINING LOOP WORKS

Let's trace one complete training step, connecting everything from previous sections:
Forward pass:

1. Embeddings (Section 3): Token IDs → vectors. Each token becomes a 4096-dimensional vector
2. Through transformer blocks (Section 4-5): The 32 transformer blocks progressively refine these vectors. Each block's attention lets tokens "look at" earlier tokens
3. Output projection (Section 6): Final hidden states → vocabulary logits

Loss computation: We know what each position should predict (the actual next token in our training data). We compute cross-entropy loss between predictions and targets:

```
# Our input: ["The", "cat", "sat", "on", "the", "mat"]
# Targets (shifted by 1): ["cat", "sat", "on", "the", "mat", END]
# For each position, compute loss:
loss_pos_0 = cross_entropy(prediction_after_"The", target="cat")
loss_pos_1 = cross_entropy(prediction_after_"The_cat", target="sat")# ... etc

total_loss = average(all_position_losses)  # One number
```

```

```

Backward pass: The loss flows backward through the network (backpropagation from Section 0). Here's the key insight: the same transformer block weights processed position 0 AND position 500. So those weights get gradient signals from both positions—they learn from all positions simultaneously.
Optimizer step: Update all 32B parameters based on the accumulated gradients, then clear gradients for the next batch.

7.4 ADAMW - SIMPLE GRADIENT DESCENT ISN'T ENOUGH

Remember from Section 0 that basic gradient descent just does:
`new_weight = old_weight - learning_rate × gradient`

This fails for large models. Here's why:
Some gradients are 0.00001 (embedding weights for rare tokens), others are 10.0 (output layer weights). With a fixed learning rate, either the small gradients never move their weights, or the large gradients explode.

Each batch is a tiny sample of all possible text. One batch's gradient might point north, the next points south. Raw gradient descent zigzags wildly.
AdamW (Adaptive Moment Estimation with Weight decay) solves both problems by maintaining two running averages per parameter:

```
# For each parameter, AdamW tracks:
momentum = 0.9 * old_momentum + 0.1 * gradient       # Smoothed gradient
variance = 0.95 * old_variance + 0.05 * gradient²    # Gradient magnitude
# Update rule:
adapted_lr = learning_rate / (sqrt(variance) + epsilon)
new_weight = old_weight - adapted_lr * momentum
# Weight decay (the 'W' in AdamW) - shrink weights toward zero:
new_weight = new_weight * (1 - weight_decay)
```

Parameters with consistently large gradients (high variance) get their learning rate automatically scaled down. Parameters with tiny gradients get scaled up. It's like giving each parameter its own custom learning rate.
Memory cost: AdamW needs to store momentum and variance for each parameter. For a 32B parameter model:

- Weights: 32B × 4 bytes = 128GB
- Momentum: 32B × 4 bytes = 128GB
- Variance: 32B × 4 bytes = 128GB
- Total: 384GB just for optimizer state (!?!?!)

7.5 LEARNING RATE WARMUP - WHY STARTING SLOW MATTERS

At initialization, model weights are random. The gradients point in essentially random directions. Taking big steps in random directions is catastrophic.
Warmup gradually increases learning rate from near-zero to your target learning rate over the first ~2000 steps:
python

```
if step < 2000:
    lr = target_lr * (step / 2000)  # Linear warmup
```

```
else:
    lr = target_lr  # Then constant (or decay)
```

During warmup, the model learns basics—which tokens commonly follow others, basic grammar. Once it has this foundation, it can safely take bigger steps.
Without warmup, especially with large batches, the initial random gradients can throw weights so far off that the model never recovers. You'll see loss explode to infinity within the first 100 steps.

7.6 MIXED PRECISION TRAINING - GETTING 2× MEMORY FOR FREE

So far we've assumed 32-bit floating point (fp32) for everything. But modern GPUs have special Tensor Cores that compute faster in 16-bit precision. Can we use half the bits without losing quality?
Brain Float 16 (bf16) is a 16-bit format designed specifically for deep learning:

- 1 sign bit, 8 exponent bits, 7 mantissa bits
- Same exponent range as fp32 (can represent 10^-38 to 10^38)
- Less precision (7 bits vs 23 bits for mantissa)

Compare to Float 16 (fp16):

- 1 sign bit, 5 exponent bits, 10 mantissa bits
- Smaller range (10^-8 to 10^4) - often causes underflow
- Slightly more precision than bf16

Mixed precision training means:

1. Store master weights in fp32 for accuracy
2. Convert to bf16 for forward pass computation
3. Compute activations and gradients in bf16
4. Convert gradients back to fp32 for weight update

```
# Without mixed precision:
hidden = transformer_block(input)  # fp32: 8 × 2048 × 4096 × 4 bytes = 268MB
# With mixed precision:
with torch.autocast(dtype=torch.bfloat16):
    hidden = transformer_block(input)  # bf16: 8 × 2048 × 4096 × 2 bytes = 134MB
```

What you save:

- Activation memory: 50% reduction (the main bottleneck)
- Memory bandwidth: 50% reduction (memory-bound ops run ~2× faster)
- Tensor Core compute: 2× faster

The noise from bf16 rounding is tiny compared to the inherent noise in SGD. In practice, bf16 training achieves identical quality to fp32.

7.7 GRADIENT ACCUMULATION - SIMULATING LARGE BATCHES ON SMALL GPUS

You want batch size 512 for stable training, but your GPU only fits batch size 8. Gradient accumulation lets you fake the large batch:
python

```
accumulation_steps = 64  # 512 / 8 = 64
for i, batch in enumerate(dataloader):
    # Forward and backward on micro-batch of 8
    loss = model(batch) / accumulation_steps  # Scale loss!
    loss.backward()  # Gradients ADD to existing gradients
if (i + 1) % accumulation_steps == 0:
        optimizer.step()      # Update weights with accumulated gradients
        optimizer.zero_grad() # Clear for next accumulation
```

Critical: You must scale loss by 1/accumulation_steps. Otherwise your gradients will be 64× too large!
This is identical to processing all 512 examples at once—gradients sum the same way. It just takes 64× longer since you process sequentially instead of in parallel.

7.8 GRADIENT CHECKPOINTING - TRADING COMPUTE FOR MEMORY

During forward pass, PyTorch normally saves all intermediate activations for the backward pass. Let's see what this means:
python

```
# Normal forward pass (saves everything):
hidden = embeddings                          # Save: 8 × 2048 × 4096 = 67MB
hidden = transformer_block_1(hidden)         # Save another 67MB
hidden = transformer_block_2(hidden)         # Save another 67MB
# ... continue for all 32 blocks
hidden = transformer_block_32(hidden)        # Total saved: 32 × 67MB = 2.1GB!
```

Gradient checkpointing (or activation checkpointing) doesn't save all these:

```
# With checkpointing - only save every 4th block:
hidden = embeddings                          # Save ✓
hidden = transformer_blocks_1_to_4(hidden)   # Save ✓ (only final output)
hidden = transformer_blocks_5_to_8(hidden)   # Save ✓
# ... etc
# During backward pass:
# Need gradient for transformer_block_6?
# Recompute: hidden_5 = forward(hidden_4), hidden_6 = forward(hidden_5)
# Now you can compute gradients
```

The trade-off:

- Memory: 4× reduction (checkpoint every 4 blocks instead of every block)
- Speed: ~30% slower (recomputing activations during backward)

But the memory savings often let you 4× your batch size, which better utilizes the GPU and ends up faster overall!

7.9 THE MEMORY HIERARCHY DURING TRAINING

For a 32B parameter model, here's what's actually in GPU memory.

Fixed costs (don't change with batch size):

- Model weights: 32B × 2 bytes (bf16) = 64GB
- Gradients: 32B × 2 bytes (bf16) = 64GB
- Optimizer state (AdamW): 32B × 8 bytes (2 × fp32) = 256GB
- Total: 384GB

Variable costs (scale with batch size and sequence length):

- Activations: batch_size × seq_len × hidden_size × num_layers × 2 bytes
- For batch=8, seq=2048: 8 × 2048 × 4096 × 32 × 2 = 4.3GB
- For batch=8, seq=65536: 8 × 65536 × 4096 × 32 × 2 = 137GB!

This is why long context is hard. A 100K context window needs more memory for activations than for the model itself! You must use gradient checkpointing or the model won't fit.

8. Distributed Training

Section 7 showed how training works on a single GPU. Reality check: a 70B parameter model needs 840GB of memory during training. The best GPU you can buy has 141GB. We need to split this across hundreds of GPUs, and how we split it determines whether our training runs in days or months—or whether it runs at all.

8.1 THE MEMORY MATH THAT FORCES DISTRIBUTION

Here's what breaks. During training, a 70B model needs:

- Weights: 140GB (70B × 2 bytes)
- Gradients: 140GB
- AdamW optimizer: 560GB (momentum + variance, both fp32)

Total: 840GB. Your H200 GPU: 141GB.
The optimizer is the killer—it's 4× larger than the model itself. This is why the jump from inference (just needs weights) to training (needs weights + gradients + optimizer) is so brutal.

8.2 DATA PARALLEL - WORKS UNTIL IT DOESN'T

Data parallel is the obvious first idea: copy the model to every GPU, give each GPU different training samples. After computing gradients, average them across all GPUs (all-reduce), then everyone updates identically.
This works beautifully for a 7B model—it needs 84GB total, fits on one GPU with room to spare. But for our 70B model? Each GPU would need the full 840GB. Data parallel is dead on arrival.

8.3 ZERO - SHARD THE STORAGE, NOT THE COMPUTATION

Microsoft's insight: after all-reduce, every GPU has identical optimizer states. Eight GPUs storing eight identical copies of a 560GB optimizer. That's wasteful.

ZeRO progressively shards these redundant parts:

- **Stage 1**: Each GPU stores only its slice of optimizer states. GPU 0 handles parameters 0-8.75B, GPU 1 handles 8.75B-17.5B, etc. Cuts optimizer memory by 8×.
- **Stage 2**: After computing gradients, don't all-reduce—do reduce-scatter so each GPU only keeps gradients for its parameters. Saves another 8×.
- **Stage 3**: Shard the model weights too. During forward pass, gather the weights you need, compute, discard. Yes, this

means gathering weights twice per layer (forward and backward), but now our 70B model fits in 105GB per GPU.

Important: ZeRO-3 still runs the complete model on every GPU. Each GPU processes different data samples, temporarily gathering whatever weights it needs. It's data parallel with clever memory management—not model parallel.

8.4 MODEL PARALLELISM - SPLIT THE COMPUTATION ITSELF

For a 650B model, even the weights alone are 1.3TB. ZeRO-3 can shard storage, but each GPU still needs to gather full layers to compute. When layers themselves are too big, you need model parallelism—split the computation, not just the storage.

Tensor Parallel splits individual matrices. That 16384×16384 attention matrix becomes eight 16384×2048 slices across eight GPUs. Each GPU computes only its slice, never seeing the full matrix. All GPUs process the same batch of data but compute different parts of the output. Problem: you need communication after every single layer. This only works within a single machine where GPUs connect via NVLink at 900GB/s.

Pipeline Parallel splits layers sequentially. GPU 0 computes transformer blocks 1-10, GPU 1 computes blocks 11-20, etc. Each GPU only ever runs its assigned blocks. Data flows through like an assembly line. The downside: pipeline bubbles where GPUs wait for data to arrive.

The key distinction from ZeRO: in model parallelism, each GPU only computes part of the model. The computation itself is distributed, not just the storage.

8.5 3D PARALLELISM - THE REAL WORLD

Nobody uses just one technique. A 650B model on 512 GPUs typically combines all three:

- TP=8 within each 8-GPU node (need that NVLink speed)
- PP=8 across 8 nodes (one pipeline stage per node)
- DP=8 data parallel replicas

Each GPU handles 650B ÷ (8×8) = 10.15B parameters—about 51GB of memory load, comfortable on a 141GB H200.

The communication hierarchy matches the hardware:

- Tensor parallel talks every layer—keeps it on fast NVLink
- Pipeline parallel talks between stages—fine over network
- Data parallel talks once per batch—works anywhere

8.6 THE TRADE-OFFS THAT MATTER

1. **Memory vs communication i**s the fundamental tension. Every technique that reduces memory increases communication. Your hardware determines what's viable—900GB/s NVLink can handle aggressive sharding, 10GB/s Ethernet can't.
2. **Hardware topology matters**. Modern clusters aren't flat—they're hierarchical. Eight GPUs per node with fast interconnect, nodes connected by slower networks. This hierarchy drives your parallelism strategy.
3. **Failure becomes routine**. With 512 GPUs at 99.9% uptime each, you have a 60% chance they're all working right now. The more complex your parallelism, the harder recovery becomes. Pure data parallel can redistribute work easily. 3D parallelism needs exact topology restoration.

8.7 WHO BUILDS WHAT

Megatron-LM (NVIDIA) pioneered efficient tensor and pipeline parallelism. The key wasn't just splitting matrices—it was fusing operations to minimize communication. Instead of gathering Q, K, V separately (3 communications), compute QKV together and gather once.

DeepSpeed (Microsoft) implements ZeRO plus memory optimizations like CPU offloading. When you need to train bigger

than your GPU memory, DeepSpeed has probably thought of it.
Megatron-DeepSpeed combines both—Megatron's model parallelism with DeepSpeed's memory optimization. This powers most 100B+ model training today.
FSDP is PyTorch's native ZeRO-3. Simpler API, fewer features. Good enough for most cases, but when you're pushing boundaries, you want DeepSpeed.

9. The Alignment Problem - Why Next-Token Prediction Isn't Enough

After all that pre-training from Sections 1-8, you have a model that's incredibly capable. It can write code, explain complex topics, translate languages, and demonstrate sophisticated reasoning. But there's a problem: the model is a simulator, not an assistant. It learned to predict what humans write next, including all the messy, harmful, and unhelpful patterns in internet text.

Let's see this concretely. You ask your freshly pre-trained model "What's the capital of France?" and instead of a clean answer, you might get:

> What's the capital of France? This is a common geography question that appears frequently on standardized tests. The answer, of course, is Paris, which has been the capital since 987 AD when Hugh Capet established the Capetian dynasty. Interestingly, many people confuse this with...

The model isn't broken—it's doing exactly what it learned. During pre-training, it saw millions of examples where someone asked "What's the capital of France?" and the text continued with explanatory context, historical background, and related trivia. The model learned that pattern perfectly. But you wanted a simple answer: "Paris."

The problem gets worse with sensitive topics. Ask "How do I hack into someone's computer?" and a pre-trained model might cheerfully provide detailed instructions, because that text pattern exists in its training data. The model has no concept that you want helpful advice, not harmful instructions. It just continues the pattern it learned.
This mismatch shows the core **alignment problem**. We trained the model to optimize for "what would humans write next" but we actually want "what would be helpful here." Those are different objectives. A human writing a hacking tutorial online is trying to demonstrate knowledge or get clicks. A human asking for help wants useful, safe guidance. Same text patterns, completely different intents.

Alignment means bridging this gap—making models that are helpful, harmless, and honest rather than just capable pattern completers. The engineering challenge is changing the training objective from "predict the next token" to "be a useful assistant," and this requires different data and training approaches.
The standard solution involves three techniques that build on each other. **Supervised Fine-Tuning (SFT)** teaches the model the format and style of helpful responses through direct examples. **Direct Preference Optimization (DPO)** trains the model to prefer better responses over worse ones by learning from human preferences. **Proximal Policy Optimization (PPO)** uses reinforcement learning to optimize complex objectives that can't be captured in simple preference pairs. Each technique solves different aspects of alignment and requires different engineering approaches.

During alignment training, we start calling the model a **policy**—a decision-maker that chooses actions (tokens) based on states (the conversation so far). This affects how we structure training data, compute losses, and measure success. The model transforms from a text predictor into a conversational agent.

10. Supervised Fine-Tuning (SFT) - Teaching Through Examples

Remember from Section 7 how we trained the model to predict next tokens? SFT uses the exact same training loop—forward pass, compute loss, backprop, optimizer step—but changes what we train on. Instead of raw internet text, we use curated conversations between users and helpful assistants..
The training data looks like this:

```
<|user|>How do I center a div in CSS?<|end|>
```

````
<|assistant|>You can center a div using flexbox:

```css
.container {
  display: flex;
  justify-content: center;
  align-items: center;
}
```<|end|>
````

Compare this to pre-training data from a CSS tutorial blog:

> "Centering divs has historically been one of the trickiest aspects of CSS. In this comprehensive guide, we'll explore multiple approaches. First, let's understand why this is challenging..."

The blog continues with background, context, and multiple examples. The SFT version teaches direct, helpful answers.

Loss Masking

The key difference from pre-training: the model only learns to predict assistant responses, not user messages. This requires loss masking:

```
sequence = ["<|user|>", "What's", "2+2", "?", "<|end|>",
           "<|assistant|>", "2+2", "equals", "4", "<|end|>"]

loss_mask = [False, False, False, False, False,  # User tokens - no loss
False, True, True, True, True]        # Assistant tokens - compute loss
```

Without masking, the model would learn to predict user messages too, getting confused about who's speaking.

TRAINING MECHANICS

The training loop from pre-training stays identical. AdamW optimizer from Section 7.4, mixed precision from 7.6, gradient accumulation from 7.7—all the same.

Main difference - SFT datasets are tiny compared to pre-training—10K to 100K conversations vs trillions of tokens. Quality beats quantity since bad examples can break the model's behavior.

Training runs much faster with higher learning rates. Why? The model already learned language during pre-training. SFT just adjusts the response style and format—like teaching someone who already speaks English to write business emails instead of text messages. Small nudges to existing weights work better than large changes.
Memory requirements are identical to pre-training: weights + gradients + AdamW optimizer state = ~384GB for a 32B model, plus activations. The only potential savings come from smaller batch sizes (since SFT datasets are small) or shorter conversation examples reducing activation memory.

BEYOND CONVERSATION STYLE

SFT teaches much more than tone. You can train:
Tool use: Teaching models to call functions by showing examples of proper API calls and responses. **Reasoning patterns**: Chain-of-thought examples where the model shows its work step-by-step. **Safety**: Examples of politely declining harmful requests. **Domain expertise:** Medical models trained on doctor-patient conversations, legal models on lawyer consultations. **Code generation:** Specific formatting for different programming languages and frameworks.

Each capability comes from examples in your SFT dataset. The model learns these patterns the same way it learned grammar

during pre-training—through gradient descent on the next-token prediction objective, just with different data.

WHAT SFT ACHIEVES

After SFT, your model behaves like an assistant. It learned the conversation format, gives direct answers, and follows the patterns from your training examples.
But SFT inherits a limitation from the next-token prediction objective (Section 7.2). You can only show one "correct" response per prompt. When there are multiple good responses—creative vs factual, detailed vs concise—SFT can't teach preferences. The cross-entropy loss from Section 7.3 treats all differences from the training example as equally wrong.
This is where preference learning becomes essential. While SFT teaches format and basic helpfulness through direct examples, the next techniques teach the model to choose better responses when multiple options exist.

11. Direct Preference Optimization (DPO) - Learning What Humans Prefer

SFT from Section 10 taught the model to be helpful through examples, but it can't handle preferences. When asked "Write a story about a robot," should the model be creative or simple? Detailed or concise? SFT can only show one "correct" answer. DPO solves this by learning from comparisons—showing the model pairs of responses and teaching it which one humans prefer.

The key insight: humans are much better at comparing responses than writing perfect ones. Ask someone to write the ideal response to "Explain machine learning" and they'll struggle. Show them two responses and ask which is better? Easy. DPO exploits this asymmetry.

Preference Data Format
DPO training data consists of response pairs with human preferences:

```
{
  "prompt": "Explain how a car engine works",
  "chosen": "A car engine converts fuel into motion through controlled explosions.
             Fuel and air mix in cylinders, spark plugs ignite the mixture,
             and the explosion pushes pistons down. These pistons turn a crankshaft,
             which ultimately rotates the wheels.",
  "rejected": "A car engine works through a complex system of interconnected
               components including the intake manifold, camshaft, valvetrain,
               connecting rods, and numerous other parts working in synchronization.
               The process begins when the intake valve opens during the intake stroke
}
```

The "chosen" response is clear and accessible. The "rejected" response is technically correct but too complex for a general audience. Through thousands of these comparisons, the model learns not just to be correct, but to match human preferences for clarity, helpfulness, and appropriate complexity.

Understanding Log Probabilities
Before diving into DPO's loss function, let's understand **log probabilities**. Remember from Section 6 how the model outputs logits, then softmax converts them to probabilities? When generating text, the model assigns a probability to each token in the sequence.
For a response like "The capital is Paris", the model computes:

- P("The" | prompt) = 0.15
- P("capital" | prompt + "The") = 0.08
- P("is" | prompt + "The capital") = 0.22

- P("Paris" | prompt + "The capital is") = 0.31

The total probability of generating this exact sequence is the product: `0.15 × 0.08 × 0.22 × 0.31 = 0.00082`
These products quickly become tiny numbers. After 100 tokens, you're multiplying 100 probabilities less than 1, giving you something like 10^-89. Computers can't handle this—it underflows to zero.
Log probabilities solve this. Instead of multiplying probabilities, we add their logarithms:

- log(P("The")) = log(0.15) = -1.90
- log(P("capital")) = log(0.08) = -2.53
- log(P("is")) = log(0.22) = -1.51
- log(P("Paris")) = log(0.31) = -1.17

Total log probability: -1.90 + -2.53 + -1.51 + -1.17 = **-7.11**
This stays numerically stable. The more negative the log probability, the less likely the sequence. A response with log probability -50 is way more likely than one with -200.

The Collapse Problem
If we just trained the model to make chosen responses more likely and rejected responses less likely, it would find a degenerate solution. Here's what would happen:

```
# Training data shows this was "chosen" for one creative writing prompt:
prompt1 = "Write a fantasy story"
chosen1 = "Once upon a time, in a land far away..."
# Model learns: "Once upon a time" = good
# So it starts using it EVERYWHERE:
prompt2 = "Write a horror story"
response2 = "Once upon a time..."  # log_prob goes from -50 to -5

prompt3 = "Write a technical manual"
response3 = "Once upon a time..."  # log_prob goes from -200 to -5

prompt4 = "Explain quantum physics"
response4 = "Once upon a time..."  # log_prob goes from -500 to -5
```

The model "collapsed"—it found that making a few phrases extremely likely for all prompts technically satisfies "make chosen responses more likely." But it destroyed the model's ability to generate appropriate, diverse responses.

The Reference Model Solution
DPO prevents collapse by comparing against a **reference model**—a frozen copy of your SFT checkpoint. The reference model remembers what reasonable language modeling looks like. Instead of optimizing "make chosen responses likely," DPO optimizes "make chosen responses *more likely than the reference model would*, but don't deviate too much."
Here's how it works in the loss function:

```
def dpo_loss(model, reference_model, prompt, chosen, rejected, beta=0.1):
    # Get log probabilities for entire sequences
    chosen_logprobs = model.log_prob(prompt, chosen)      # e.g., -45.2
    rejected_logprobs = model.log_prob(prompt, rejected)  # e.g., -62.8

    # Get reference model's log probabilities (frozen SFT model)
    # This is what the model USED to think before preference training
```

```
    with torch.no_grad():
            ref_chosen_logprobs = reference_model.log_prob(prompt, chosen)    # -48.1
            ref_rejected_logprobs = reference_model.log_prob(prompt, rejected) # -61.2

    # Compute how much current model differs from reference
    # Positive = model likes it MORE than reference did
    # Negative = model likes it LESS than reference did
    chosen_log_ratio = chosen_logprobs - ref_chosen_logprobs      # -45.2 - (-48.1) =
    rejected_log_ratio = rejected_logprobs - ref_rejected_logprobs # -62.8 - (-61.2) =

    # DPO loss: make chosen more likely than rejected, but relative to reference
    # This prevents collapse - can't just make everything extremely likely
    loss = -torch.log(torch.sigmoid(beta * (chosen_log_ratio - rejected_log_ratio)))
    return loss
```

Now if the model tries to use "Once upon a time..." for "Explain quantum physics," the reference model says "that had log probability -500 before!" The huge deviation gets penalized. The model learns to improve preferences while staying in the realm of reasonable responses.
The `beta` parameter controls this tradeoff. Small beta (0.1) means stay very close to the SFT model. Large beta (0.5) means stronger preference learning but risk of degrading response quality.

Training Mechanics
DPO training looks similar to SFT but processes pairs:

```
for batch in preference_dataset:
    # Each batch contains (prompt, chosen, rejected) triples
    loss = 0
for prompt, chosen, rejected in batch:
        loss += dpo_loss(model, reference_model, prompt, chosen, rejected)

    loss.backward()
    optimizer.step()  # Same AdamW from Section 7.4
    optimizer.zero_grad()
```

Memory requirements: You need two models in memory (training model + frozen reference), so roughly 2× the model size plus optimizer states. For our 32B model:

- Training model: 64GB
- Reference model: 64GB (frozen, no gradients)
- Gradients: 64GB
- AdamW state: 256GB
- Total: ~448GB plus activations

Training typically uses smaller batches than pre-training since preference datasets are smaller (usually 10K-100K preference pairs). The model converges quickly—often within a few thousand steps.

What DPO Achieves
After DPO, the model makes better choices. It learned from thousands of comparisons which responses humans prefer. The model becomes more helpful, clearer, and better calibrated to human preferences.
DPO works especially well for subjective qualities that are hard to specify but easy to recognize: writing style, appropriate detail level, tone, and helpfulness. The model learns these preferences implicitly from the comparison data.
But DPO has limits. It assumes preferences are consistent and can be captured by comparing pairs. For complex multi-objective optimization—like being helpful while avoiding specific harms, or optimizing for different user populations

simultaneously—you need more sophisticated approaches. That's where PPO and full reinforcement learning become necessary.